

DEPLOYMENT HANDBOOK

# Regulatory Writing & Intelligence Deploying on AWS

---

Step-by-step, console and CLI — from an empty AWS account to a running, governed, human-gated agent. Tailored edition: AgentId **01-regulatory-writing**; approval gate **REGULATORY\_APPROVER**.

---

Systems-Integrator field & delivery reference · June 2026

*Illustrative reference — figures are mockups, not actual AWS screenshots. Validate for your environment.*

**Tailored edition for Regulatory Writing & Intelligence** (AgentId 01-regulatory-writing ). This is the companion to the suite master handbook ( docs/DEPLOYMENT-HANDBOOK.md ); it carries this agent's systems, approver role, workflow, and console figures. The phases and controls are identical across all eight agents.

0. Two deployment shapes

| Shape                         | What runs                                                                                                                                                          | When to choose                                                      |
|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| Container lift                | The LangGraph agent image on <b>Bedrock AgentCore Runtime</b> (or ECS Fargate)                                                                                     | Fastest; agent code unchanged                                       |
| Native rebuild<br>(this book) | Deterministic steps in <b>Lambda</b> , drafting via <b>Strands/Bedrock</b> , orchestrated by <b>Step Functions</b> with a <code>waitForTaskToken</code> human gate | Highest fidelity to managed-serverless; clearest audit & HITL story |

Both deploy through the same CloudFormation quickstart with `AgentId=01-regulatory-writing` , and both put every system-of-record call behind the **MCP authorization gateway = Bedrock AgentCore Gateway + Identity**.

**This agent at a glance** — systems: this edition connects RIM and related systems; the **approval gate** is the **REGULATORY\_APPROVER**; high-risk tools requiring human approval: `rim.create_submission_draft` , `dms.put_draft` . Time budget: ~2–4 hours for a first dev deployment once prerequisites are granted.

## 1. Prerequisites

| You need                                                     | Who provides it        | Notes                                                           |
|--------------------------------------------------------------|------------------------|-----------------------------------------------------------------|
| AWS account/sandbox with admin (or the scoped deploy policy) | Customer cloud team    | Start in dev, never prod                                        |
| <b>Amazon Bedrock model access</b> (Claude)                  | Account admin          | One-time per account+region (Phase 3.1)                         |
| An <b>AgentCore-enabled Region</b>                           | Architect              | e.g. <code>us-east-1</code>                                     |
| Enterprise <b>IdP</b> metadata (Okta/Entra/AD)               | Customer identity team | SAML/OIDC metadata URL                                          |
| The RIM endpoint + an API token                              | Customer system owner  | Deploy against the bundled reference service first, then swap   |
| Local tooling                                                | You                    | AWS CLI v2, Docker/Finch (ARM64), Python 3.11, <code>zip</code> |

**Confirm before you touch the console:** who is the named **REGULATORY\_APPROVER** for this agent's human gate, and who signs computer-system validation at go-live.

## 2. Pre-flight (local)

```
aws sts get-caller-identity           # right account?
cd 01-regulatory-writing-agent && python -m venv venv && . venv/bin/activate
pip install -r requirements.txt && pip install -e ../platform_core
EXTRACT_MODE=demo pytest tests/ -q    # the agent is healthy offline first
```

## 3. Phase 1 — Account foundation

### 3.1 Enable Bedrock model access (Console)

Bedrock → **Model access** → **Manage model access** → check the Anthropic Claude tiers → **Save**; wait for **Access granted**. Confirm the Region (top-right) is AgentCore-enabled.

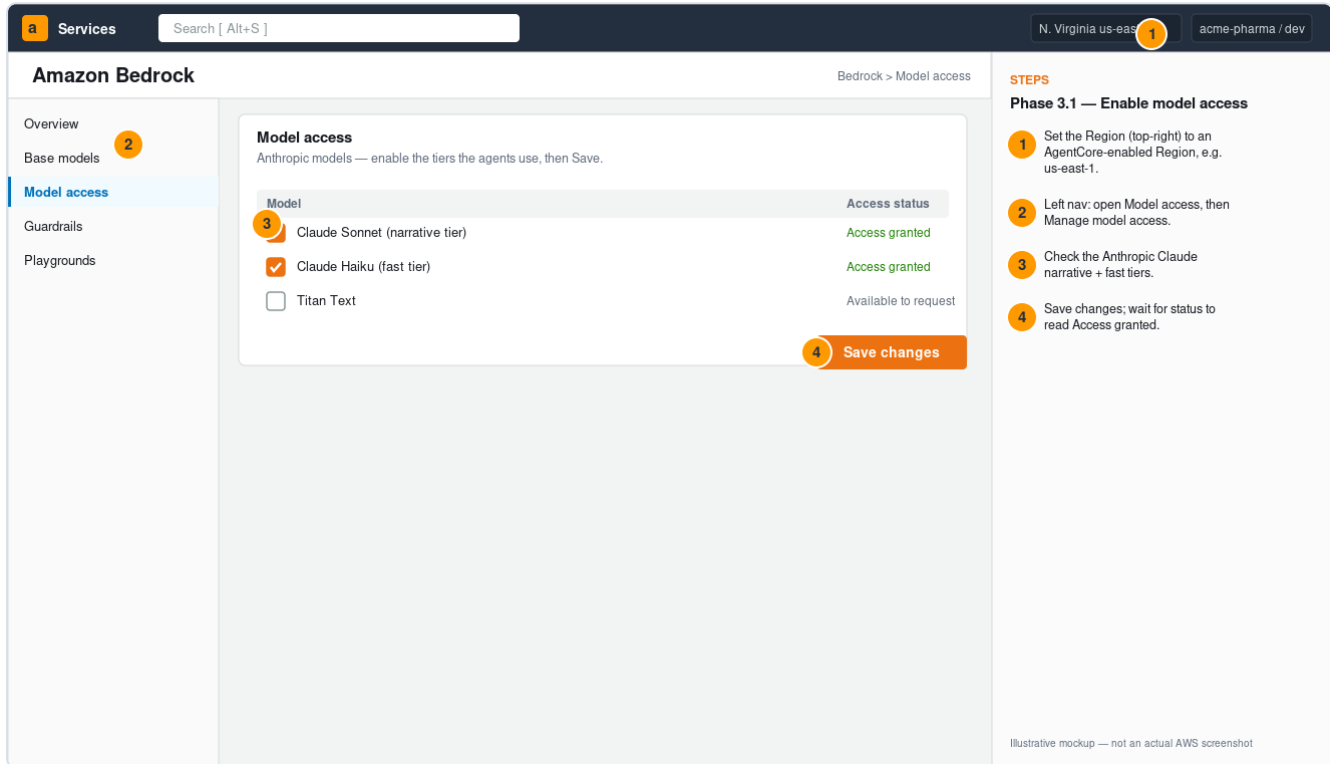


Figure 1. Amazon Bedrock → Model access (illustrative mockup).

### 3.2 Create a Bedrock Guardrail (the CloudFormation quickstart creates this)

PHI filters (SSN→Block; Email/Name/Phone→Anonymize) + a denied topic for off-label/absolute claims.  
Record the **Guardrail ID**.

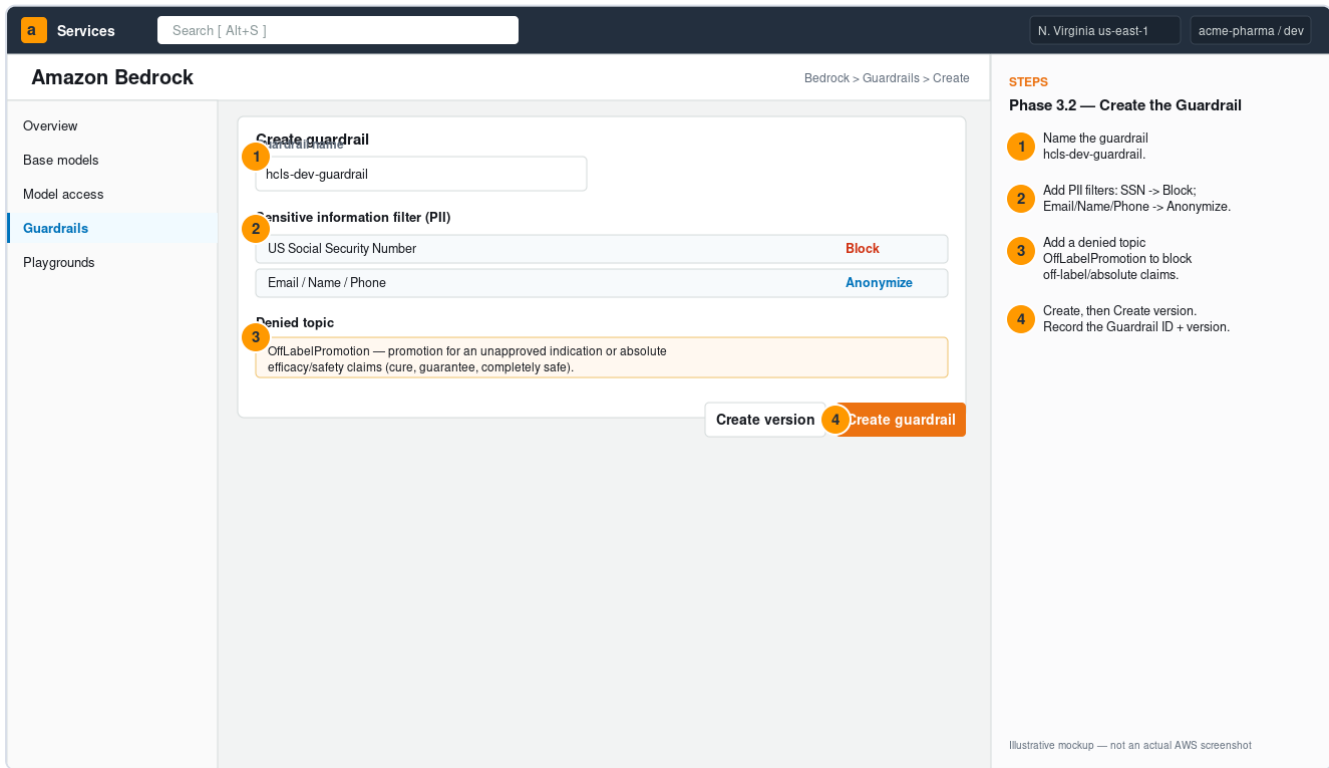


Figure 2. Bedrock → Guardrails (illustrative mockup).

### 3.3 Cognito + IdP federation — map the approver

Federate the customer IdP and map the approver group to `custom:hcls_role`. For this agent, map `GRP-REG-APPROVERS` → `REGULATORY_APPROVER`.

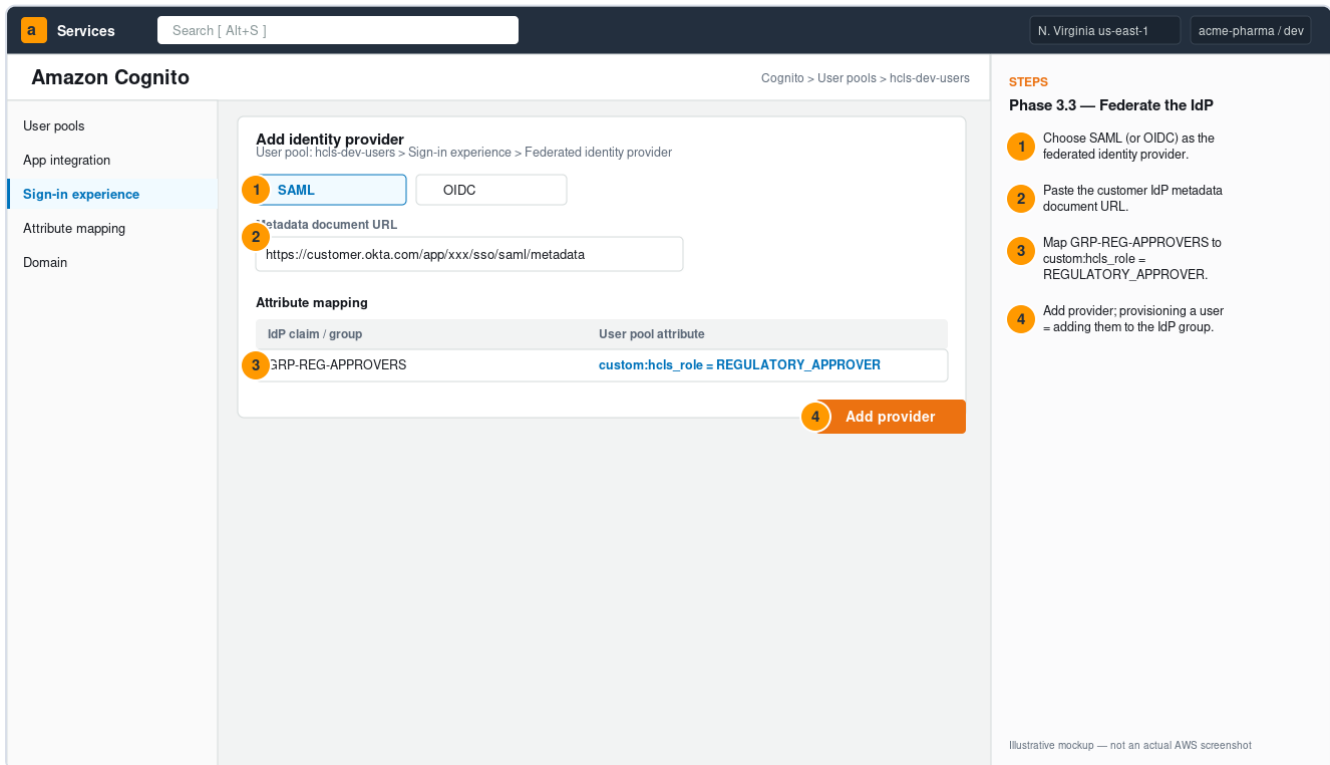


Figure 3. Cognito → Sign-in experience: map GRP-REG-APPROVERS → REGULATORY\_APPROVER (illustrative mockup).

**3.4 KMS key (quickstart creates `alias/hcls-dev`) — per-environment CMK encrypting the audit table, review table, and WORM bucket.**

## 4. Phase 2 — Stage artifacts

```
aws s3 mb s3://my-cfn-bucket
aws s3 cp infra/cloudformation/ s3://my-cfn-bucket/hcls/ --recursive --exclude "*" --include "*.yaml"
cd aws-native-reference/01-regulatory-writing && mkdir -p build
zip -r build/lambda.zip core.py strands_agent.py lambda/ requirements.txt
aws s3 mb s3://my-code-bucket
aws s3 cp build/lambda.zip s3://my-code-bucket/01-regulatory-writing/lambda.zip
```

(Container path: build the ARM64 image from `aws-native-reference/_shared/runtime` and push to ECR.)



## 5. Phase 3 — Deploy the infrastructure (CloudFormation)

CloudFormation → **Create stack** → template S3 URL → set parameters (**AgentId** **01-regulatory-writing** , Environment, DeployMode, code bucket, IdP URL) → check the IAM capability box → **Submit** → watch Events to **CREATE\_COMPLETE** → read **Outputs** ( **GatewayId** , **GuardrailId** , **AuditTable** ).

The mockup shows the AWS CloudFormation 'Create stack' console. The left sidebar has a 'Stacks' section with 'Create stack' selected. The main content area is titled 'Create stack' and contains four numbered steps: 1. Specify template: the quickstart.yaml S3 URL (https://my-cfn-bucket.s3.amazonaws.com/hcls/quickstart.yaml); 2. Set AgentId = 01-regulatory-writing (plus Environment, DeployMode, code bucket, IdP URL); 3. Check the IAM capability acknowledgement (checked); 4. Submit; watch Events to CREATE\_COMPLETE; read Outputs. The right sidebar shows 'STEPS' and 'Phase 5 — Deploy with CloudFormation'.

Figure 4. CloudFormation → Create stack for AgentId 01-regulatory-writing (illustrative mockup).

```
aws cloudformation deploy --template-file infra/cloudformation/quickstart.yaml \
  --stack-name hcls-dev-01-regulatory-writing --capabilities CAPABILITY_NAMED_IAM \
  --parameter-overrides Environment=dev AgentId=01-regulatory-writing DeployMode=native \
  TemplateBaseUrl=https://my-cfn-bucket.s3.amazonaws.com/hcls \
  LambdaCodeBucket=my-code-bucket LambdaCodeKey=01-regulatory-writing/lambda.zip \
  IdpMetadataUrl=https://customer.okta.com/app/xxx/sso/saml/metadata
```

Set `Environment=prod` in production: the Guardrail becomes mandatory (the LLM factory refuses to start un-guardrailed).

## 6. Phase 4 — The MCP layer (AgentCore Gateway + Identity)

The `agentcore-gateway.yaml` nested stack created the Gateway (JWT-authorized by your Cognito pool) and one **target per system of record** this agent uses. High-risk targets ( `rim.create_submission_draft`, `dms.put_draft` ) require a human-approval token — enforced by the `waitForTaskToken` gate in Phase 8. Deny-by-default authorization (agent grant  $\cap$  user entitlement), scoped tokens, and PHI-masked audit match the Python reference in `platform_core`.

## 7. Phase 5 — Configure (connect RIM)

### 7.1 Store the credential (Secrets Manager)

Store the RIM API token as `hcls/rim_api_token` (key `rim_api_token`). The `hcls/` prefix is what `get_secret()` resolves; the agent role has `GetSecretValue` on `hcls/*`.

The screenshot shows the AWS Secrets Manager console. The main panel is titled 'Store a new secret' and 'Secret type' is set to 'Other type of secret'. Under 'Key / value', the key is 'rim\_api\_token' and the value is masked with '\*\*\*\*\* (RIM API token)'. The 'Secret name' is 'hcls/rim\_api\_token'. A note at the bottom states: 'The hcls/ prefix is what get\_secret() resolves; the agent IAM role has GetSecretValue on hcls/\*'. A 'Store' button is at the bottom right. The right sidebar shows a 'STEPS' section with four steps: 1. Choose Other type of secret, 2. Add key rim\_api\_token = the RIM API token, 3. Name the secret hcls/rim\_api\_token (the hcls/ prefix matters), 4. Store. The agent role has GetSecretValue on hcls/\*.

Figure 5. Secrets Manager → Store `hcls/rim_api_token` (illustrative mockup).

### 7.2 Point at live inference + live systems

| Variable                          | Value                       | Effect                                |
|-----------------------------------|-----------------------------|---------------------------------------|
| <code>LLM_PROVIDER</code>         | <code>bedrock</code>        | In-account inference (no data egress) |
| <code>BEDROCK_GUARDRAIL_ID</code> | from stack Outputs          | PHI + off-label filter                |
| <code>CONNECTOR_MODE</code>       | <code>live</code>           | Real connectors, not fixtures         |
| <code>RIM_BASE_URL</code>         | the RIM base URL            | Where the live connector calls        |
| <code>ENVIRONMENT</code>          | <code>prod</code> (in prod) | Makes the Guardrail mandatory         |

This agent's systems / connectors:

| kind | System                     | Base URL env |
|------|----------------------------|--------------|
| rim  | Veeva Vault RIM            | RIM_BASE_URL |
| dms  | Veeva Vault / OpenText DMS | DMS_BASE_URL |

**Tip:** deploy first with RIM\_BASE\_URL pointed at a reference/staging endpoint to validate the pipeline, then swap to the customer's production RIM — a one-variable change, no code change.

## 8. Phase 6 — Smoke test + the human gate

Step Functions → open `hcls-dev-01-regulatory-writing` → **Start execution** with the agent's `sample_input.json`. The graph runs **Assemble** → **Draft** → **Check** → **RouteChoice** → **HumanReviewGate** → **Finalize** and **pauses at HumanReviewGate** — the `waitForTaskToken` gate for the **REGULATORY\_APPROVER**. It is supposed to wait; nothing finalizes without a human.

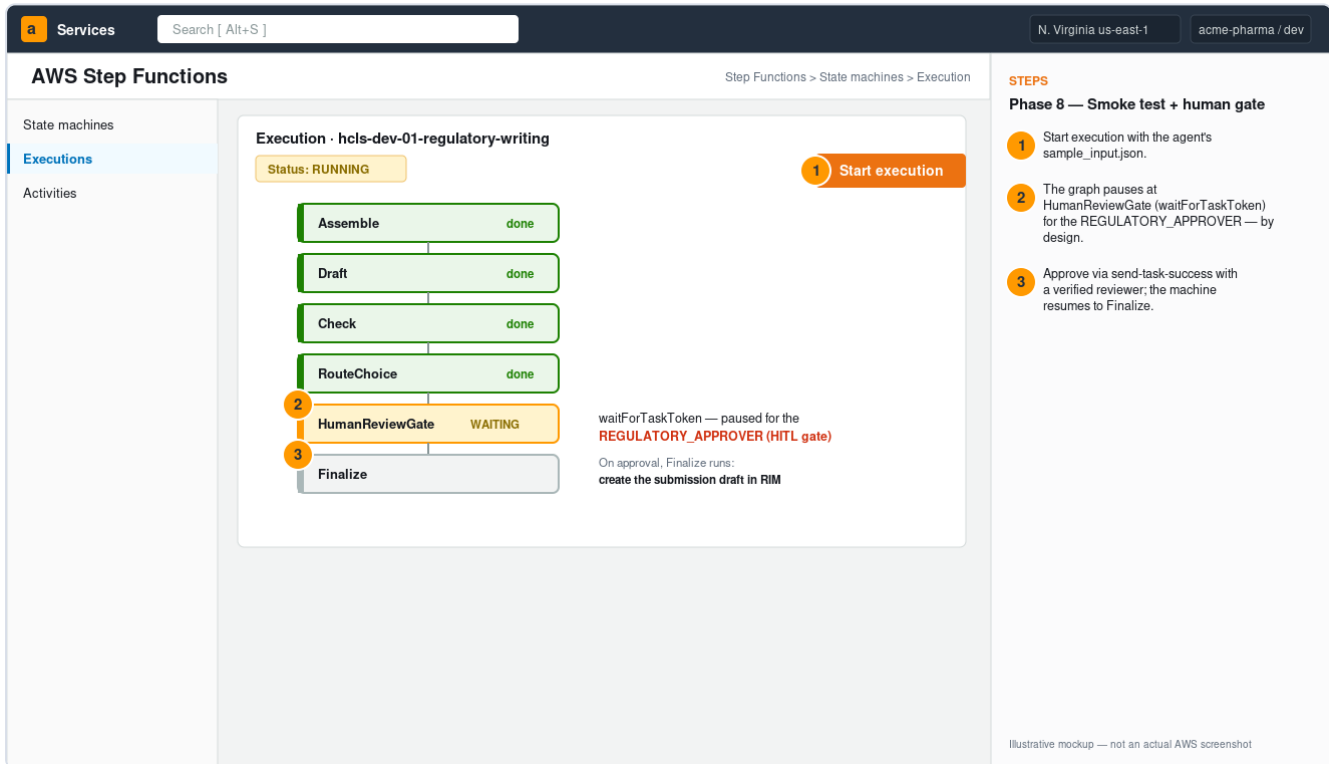


Figure 6. Execution paused at HumanReviewGate for the **REGULATORY\_APPROVER** (illustrative mockup).

```
aws stepfunctions send-task-success --task-token "<token-from-review-table>" \
  --task-output '{"approved": true, "reviewer": {"sub": "approver-1",
"custom:hcls_role": "REGULATORY_APPROVER"}}'
```

On approval the machine resumes to **Finalize**, which will create the submission draft in RIM (gateway-authorized) and seal the audit trail. To reject, use `send-task-failure`.

### 8.3 Confirm the audit trail (DynamoDB)

Open `hcls-dev-audit` → **Explore items**: append-only, PHI-masked entries per step, with the **REGULATORY\_APPROVER** bound to the approved action. This is your 21 CFR Part 11 evidence.

Services

Search [ Alt+S ]

N. Virginia us-east-1

acme-pharma / dev

Amazon DynamoDB

DynamoDB > Tables > hcls-dev-audit > Explore items

Tables

Explore items

PartiQL editor

Backups

Explore items · hcls-dev-audit

| ts       | d <sup>1</sup> n | tool                        | user     | args (PHI-masked)    |
|----------|------------------|-----------------------------|----------|----------------------|
| 16:31:02 | ALLOW            | rim.get_obligations         | u-auth-1 | product=Demo-Drug    |
| 16:31:06 | ALLOW            | dms.get_document            | u-auth-1 | doc_id=CSR-301       |
| 16:31:10 | PENDING_APPROVAL | rim.create_submission_draft | u-auth-1 | awaiting approver    |
| 16:36:14 | ALLOW            | rim.create_submission_draft | u-appr-3 | approved_by=u-appr-3 |

Append-only by IAM policy (UpdateItem/DeleteItem denied) — tamper-evident, 21 CFR Part 11.

STEPS

Phase 8.3 — Confirm the audit trail

1

Each workflow step writes an append-only, PHI-masked audit item.

2

Note the high-risk action gated as PENDING\_APPROVAL before the human approves.

3

After approval the action is ALLOW with approved\_by bound to the reviewer.

Illustrative mockup — not an actual AWS screenshot

Figure 7. Append-only, PHI-masked audit trail (illustrative mockup).

## 9. Go-live checklist

- [ ] Bedrock model access granted; Guardrail attached and tested (PHI string blocked/anonymized).
- [ ] IdP federation live; `GRP-REG-APPROVERS` maps to `REGULATORY_APPROVER`; a non-approver is correctly denied the high-risk action.
- [ ] `CONNECTOR_MODE=live` against RIM; a read and a (test) write both appear in the audit trail.
- [ ] HITL gate verified: execution pauses; finalize only after `send-task-success` with a verified `REGULATORY_APPROVER`.
- [ ] Audit trail append-only (update/delete denied); WORM retention set.
- [ ] `pytest` + governance evals captured as validation evidence; prompt manifest matches.
- [ ] Customer CSV/CSA signed for the intended use; penetration test scheduled/done.
- [ ] Runbooks reviewed: incident, DR, HITL-queue, model-degradation.

## 10. Troubleshooting

| Symptom                                              | Likely cause                                                                  | Fix                                                                                                   |
|------------------------------------------------------|-------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| <code>AccessDeniedException</code> on Bedrock        | Model access not granted in Region                                            | Phase 3.1                                                                                             |
| Stack fails on <code>AWS::BedrockAgentCore::*</code> | Region lacks AgentCore                                                        | Use an AgentCore Region, or the API Gateway + Lambda-authorizer + Cognito fallback                    |
| LLM factory refuses to start                         | <code>ENVIRONMENT=prod</code> but no <code>BEDROCK_GUARDRAIL_ID</code>        | Set the Guardrail ID (intentional guard)                                                              |
| Execution stuck at <code>HumanReviewGate</code>      | Working as designed                                                           | Send <code>send-task-success</code> with a verified <code>REGULATORY_APPROVER</code>                  |
| Connector <code>NotImplementedError</code>           | <code>CONNECTOR_MODE=live</code> but that system's live connector isn't wired | Implement typed methods in <code>connectors/live.py</code> (mirror <code>LiveSafetyConnector</code> ) |
| Tool call denied unexpectedly                        | User role lacks entitlement / agent not granted tool                          | Check <code>ROLE_ENTITLEMENTS</code> / <code>AGENT_TOOL_GRANTS</code> + Cognito mapping               |

Companion to the suite master handbook ( [docs/DEPLOYMENT-HANDBOOK.md](#) ), [SOLUTION-FIELD-GUIDE.md](#) (sales/SA), and this agent's [docs/](#) set. For operations after go-live, see [runbooks/](#).